The Big Picture: What Are We Trying to Do?

Imagine proteins as tiny machines made of chains that fold into specific 3D shapes. When they fold incorrectly, it can cause diseases like Alzheimer's or Parkinson's. For decades, scientists have tried to understand **how** proteins fold so reliably.

Think of it like this: You have a 100-piece jigsaw puzzle that can assemble itself perfectly in seconds. How does it know where every piece goes?

Our project is building a more realistic computer simulation to watch this folding process and discover **temporary stopping points** (intermediates) that previous simulations missed.

The Foundation Papers

Clementi Paper (The "Go Model" - 2000s)

The Simple Idea: "Let's assume the final folded structure is perfect, and only pieces that touch in the final structure should stick together."

- **Like a "perfect" puzzle:** Pieces only connect if they're neighbors in the final picture
- **Very simplified:** All interactions have the same strength
- **Problem:** Real proteins aren't this perfect - they sometimes get stuck in temporary shapes

Cho Paper (The "Flavored Go Model" - ~2019)

The Upgrade: "Different amino acids have different chemical personalities - some are oily, some are charged, some are neutral. Let's account for that."

- **Adds "flavor":** An oily piece (like Leucine) sticks more strongly to other oily pieces than to charged pieces
- **Uses real data:** The MJ matrix contains interaction strengths measured from thousands of real proteins
- **Still missing:** Even with flavors, the model only allows "correct" interactions

Our Revolutionary Step: Adding "Mistakes"

Our Innovation: "In real folding, proteins make temporary 'mistakes' - wrong pieces stick together briefly before finding their correct partners. Let's simulate that!"

This is what makes our project potentially Nobel-worthy - we're adding biological realism that could reveal completely new folding pathways.

Your Step-by-Step Execution Plan

Phase 1: Understand and Prepare

What you're doing: Gathering your tools and materials

Why this matters:

- **PDB file:** This is the "answer key" - the final folded structure from experiments
- **MJ matrix:** This is your "flavor guide" telling you which amino acids like to interact
- **Scale factor f = 0.05:** This controls how strong the "wrong" attractions are (5% as strong as "correct" ones)

Real-world analogy: You're a chef gathering recipes (MJ matrix), ingredients (PDB structure), and deciding how much seasoning to use (f=0.05).

Phase 2: Define "Contacts" Cleanly

What you're doing: Making two lists - "correct friends" and "potential wrong friends"

Why this matters:

- **Native contacts:** Pairs that SHOULD stick together in the final structure
- **Non-native candidates:** Pairs that COULD temporarily stick together during folding

How it works:

- Look at your PDB structure
- Measure distances between all amino acids
- If two are close (< 6.5 Å) and not neighbors in the chain → "correct friends"
- All other possible pairs → "potential wrong friends"

Real-world analogy: At a party, you list who's in the final photo (native contacts) and who might chat briefly before finding their real friends (non-native candidates).

Phase 3: Decide Representation

Critical Decision Point: How detailed should our "wrong attractions" be?

Option A: Full Specificity (Recommended)

- Every possible "wrong pair" gets its own custom attraction strength
- **Pro:** Maximum realism
- **Con:** Lots of data to manage

Option B: Class-Level

- Group amino acids into "oily", "charged", "neutral"
- All oily-oily wrong pairs have the same attraction strength
- **Pro:** Simpler
- **Con:** Less realistic

Why this choice matters: This determines whether your simulation can discover subtle intermediates that depend on specific amino acid interactions.

Phase 4: Compute Non-Native Strengths

What you're doing: Calculating exactly how strong each "wrong attraction" should be

The Math (Don't worry - I'll explain):

text

For each wrong pair (i,j):
  Look up their natural attraction from MJ matrix: ε_MJ
  Scale it down: ε_wrong = 0.05 × ε_MJ
  Convert to simulation parameters: C6 and C12

Why the clipping? Some amino acids REALLY love each other (very negative MJ values). We cap these so one super-strong wrong attraction doesn't ruin everything.

Real-world analogy: You're deciding exactly how strongly each "wrong conversation" at the party should pull people away from their real friends.

Phase 5: Write Topologies

What you're doing: Creating the instruction files for your simulation

You'll generate multiple versions:

- f=0.top: No wrong attractions (baseline)
- f=0.01.top: Very weak wrong attractions
- f=0.05.top: Medium wrong attractions
- f=0.10.top: Stronger wrong attractions

Why multiple versions? This lets you see how different levels of "mistakes" affect folding.

Phase 6: Sanity Checks

What you're doing: Making sure your simulation doesn't explode!

Quick tests:

- Does the protein immediately collapse into a ball?
- Do energies go to infinity?
- Does the folded structure remain stable?

Why this is crucial: A small math error could make the simulation unrealistic. Better to catch it now than after running for weeks.

Phase 7: Production Sampling

What you're doing: Running the actual experiments

You'll run:

- 8+ simulations for each f-value
- From both folded and unfolded starting points
- For enough time to see multiple folding/unfolding events

Why so many repeats? Protein folding is random - you need statistics to be sure your results aren't flukes.

Phase 8: Compute Observables

What you're doing: Turning raw simulation data into understandable metrics

Key measurements:

- **Q(t):** What fraction of "correct contacts" are formed over time
- **RMSD:** How different is the current shape from the final folded structure
- **Rg:** How compact is the protein
- **Contact maps:** Which specific pairs are interacting

Real-world analogy: You're taking multiple measurements of the puzzle assembly process - how many correct connections, how close to the final picture, how compact the pieces are.

Phase 9: Build Free-Energy Landscapes

The Grand Finale - Where Discoveries Happen

What you're doing: Creating a "map" of all possible protein shapes and how stable they are

How it works:

- Combine your Q, RMSD, and Rg measurements
- Calculate the "free energy" - lower energy = more stable
- Create a topographic map where valleys are stable states and peaks are unstable states

What you're looking for:

- **Vanilla model:** One deep valley (the native state)
- **Our model with f > 0:** NEW valleys appear! These are your discovered intermediates

The "Aha!" moment: When you see a new stable valley that only appears when you add non-native interactions. This represents a folding intermediate that no one has seen before.

What Could This Discover?

1. **New folding pathways:** Proteins might take detours we never knew about
2. **Misfolding mechanisms:** How proteins get stuck in disease-causing shapes
3. **Design principles:** How to engineer proteins that fold better
4. **Fundamental physics:** New insights into how complexity emerges from simple rules